

CS 5330 Final Project Brainstorm

Thomas Kulch, Darshan Kedari, Wade Williams

CS 5330: Computer Vision & Pattern Recognition

Final Project Brainstorm — Object Detection for Collision Avoidance, in Detail

Summer 2026

#	Idea	Data / platform	Headline metric	Training add-on (optional)	Effort	Risk
1	Bird's Eye View (BEV) obstacle mapping (proposal)	KITTI + SUN RGB-D	Centroid error by distance; 3D AP; mAP@0.25	Detector FT · depth-head FT · frustum head	~12-15 pd	Low-med
2	360° surround-view mapping	nuScenes-mini	Official center-distance mAP	Detector FT on nuImages	+2-4 pd	Low
3	Persistent room-scale map	ScanNet / TUM RGB-D	Occupancy P/R; object recall	Indoor detector FT	+4-6 pd	Med
4	Navigation-in-the-loop (extension #2)	Habitat-Sim	Success Rate + SPL vs oracle	Behavior cloning from oracle	+5-8 pd	Med
5	Robot-stack deployment	Gazebo + ROS 2 + Nav2	BARN score (success × time)	— (no natural fit)	+6-10 pd	High
6	Dynamic obstacles + forecasting	KITTI tracking + MOT17	MOTA/IDF1; ADE/FDE; alarm P/R	DeepSORT Re-ID · learned forecaster	+5-7 pd	Med
7	Language-directed navigation	Own recordings + Habitat	Goal-reach success + SPL	Few-shot YOLO-World FT	+4-6 pd	Med

pd = person-days; our budget is 3 people × 3 weeks ≈ 45 pd, ~12 consumed by report + video. FT = fine-tune.

1. Bird's Eye View (BEV) Obstacle Mapping from Detection + Depth (KITTI + SUN RGB-D) — *our proposal*

Idea: Convert each frame of RGB-D (or RGB + estimated-depth) video into a top-down metric obstacle map that labels every obstacle's class and 3D position and raises near-field collision warnings, evaluated outdoors on [KITTI](#) and indoors on [SUN RGB-D](#).

Data / Datasets

- [KITTI 3D object detection](#): 7,481 labeled frames (Car/Pedestrian/Cyclist 3D boxes); 12 GB of images plus calib and labels; no registration via [AWS Open Data](#) (`s3://avg-kitti`) or the

[Kaggle mirror](#) (skip the 29 GB LiDAR). Standard 3,712 train / 3,769 val split.

- [SUN RGB-D](#): 10,335 real RGB-D frames (Kinect/Xtion/RealSense) with 3D boxes; 6.4 GB direct from Princeton; use the pre-extracted [Hugging Face mirror](#) to skip the official MATLAB extraction step.
- Our own RealSense recordings (extension #1): handheld walkthroughs + one static scene with obstacles at tape-measured 1–4 m distances ([pyrealsense2](#) wheels now work on Ubuntu 24.04).
- Swap option: KITTI can be replaced by [nuScenes](#) as the outdoor benchmark — nuScenes-mini (3.9 GB, [direct download](#), no account) runs the identical pipeline with the friendlier official center-distance metric, or the full set (~60 GB keyframes-only / ~314 GB complete) if we want scale; details and trade-offs under idea 2.

Methods / Models

- [YOLO11n/s](#), pretrained on COCO, exported to ONNX (~18/11 fps CPU at 640 px per the [official benchmarks](#)) — 2D detection only.
- [Depth Anything V2](#) *metric* checkpoints: [Outdoor/VKITTI](#) (80 m) for KITTI, [Indoor/Hypersim](#) (20 m) for indoor frames; run in PyTorch on GPU (the metric ONNX export has a [known accuracy bug](#)).
- [RANSAC](#) plane fitting — implemented by us (~40 lines of numpy) for ground removal.
- Euclidean clustering / [DBSCAN](#) — implemented by us — for obstacle extraction.
- Training: none required (see Training add-on).

Pipeline

- Preprocess: load frame + camera intrinsics (dataset calib files / RealSense API); resize to 640 px for the detector; align depth to color (sensor SDK does this; DA-V2 output is already aligned).
- Detect: YOLO → 2D boxes + classes; map COCO classes to dataset classes (stated explicitly).
- Depth: sensor depth (SUN RGB-D, RealSense) or DA-V2 metric depth (KITTI).
- Back-project: pinhole model $X = (u - cx) \cdot Z / fx$, $Y = (v - cy) \cdot Z / fy$ → metric 3D point cloud.
- Ground removal: RANSAC plane fit, delete inliers (floor/road).
- Cluster: group remaining points into obstacle blobs — catches objects YOLO doesn't know.
- Fuse: project each cluster into the image; overlap with a YOLO box → class label, else "unknown obstacle."
- Render: BEV occupancy grid with class-colored markers, distance rings, red warning zone (<1.5 m ahead).

Evaluation

- KITTI: 3D centroid error binned by range (0–10/10–20/20–30 m) via the [distance-binned fork](#) of [kitti-object-eval-python](#), plus 3D AP at relaxed IoU; context: published [pseudo-LiDAR](#) pipelines score $\approx 12/10/10$ AP3D (E/M/H @ IoU 0.7).
- SUN RGB-D: standard 10-class mAP @ 3D IoU 0.25 ([VoteNet](#) protocol; learned-method ceiling ≈ 57).

- Own recordings: absolute distance error vs tape measure; RealSense depth vs DA-V2 depth on identical frames.

Training add-on (optional — pick at most one; zero-shot stays the baseline row; train on train splits only)

- Option A — fine-tune the detector (safest; ~1-2 h GPU; +3 pd): convert KITTI train-split labels to YOLO format (~50-line script), then `yolo train model=yolo11n.pt data=kitti.yaml epochs=50 imgsz=640` (batch $\approx$ 16, AMP). Indoor analog on SUN RGB-D's 2D boxes. Ablation: does better 2D detection propagate to better 3D localization, or is depth the bottleneck? (Ties directly to [MonoDLE](#)'s localization-error argument.)
- Option B — fine-tune the metric depth head (most on-theme; +4-5 pd): DA-V2's outdoor head was tuned on *Virtual* KITTI and shows scale bias on real KITTI — fine-tune the ViT-S metric head on 1-2k real KITTI frames against sparse LiDAR depth (L1 on valid pixels), then measure whether the bias and the downstream centroid error shrink.
- Option C — learned frustum head (most interesting; +4-5 pd): keep the detector and depth, but train a small PointNet (~1M params) to regress each detection's 3D centroid + extent from its frustum points, supervised by KITTI 3D labels — a miniature [Frustum PointNets](#). Star ablation: classical RANSAC + clustering vs learned head, same detector, same depth.

Closed-loop steering (optional — three ways to make something act on the commands)

1. Simulated agent — this is literally our extension #2. Habitat closes the loop properly: the agent steps, the observation changes, our map updates, A* replans. It's also the only version with rigorous, citable metrics (Success Rate, SPL vs the geodesic oracle). If "closed-loop steering in project 1" is the goal, the honest answer is: we already planned it — it's the Habitat extension, and nothing about idea 1's core needs to change.
2. A human as the actuator (+2-3 pd) — the cheap live demo. Live RealSense feed, the steering policy computes the free-gap heading, and the system emits cues — stereo audio panning (beep in the left ear = steer left) or arrows on a screen. A teammate carrying the rig follows the cues through an obstacle course. This is genuinely closed-loop: the human executes the command, the camera's next view reflects it. Evaluate with course-completion rate and collision count over N runs, cues-on vs cues-off. It's essentially an assistive-device design, and it makes a fantastic 30 seconds of presentation video. (Safety: sighted carrier, spotter, soft obstacles.)
3. A real robot (extension #3 territory, +many pd). The principled route is the Gazebo/Nav2 path from idea 5 — our obstacle points feed a costmap, Nav2 does DWA-style control — proven in sim first, then pointed at the real camera on a TurtleBot-class base (or, post-course, the lab's Go2 via its velocity-command SDK). This is the only version where the guidance is firm: do not put it anywhere near the core. Integration debugging is unbounded, and the course grades our report and evaluation, not our robot.

3D scene reconstruction add-on (optional — a demo garnish, not core)

- The recipe everywhere is poses + fusion: we already produce per-frame 3D point clouds, and [Open3D's TSDF integration](#) fuses posed depth frames and extracts a mesh with marching cubes in ~50 lines. The pose source is the only hard part.
- KITTI: the raw sequences ship GPS/IMU (OXTS) poses — transform each frame's cloud into the world frame and accumulate a street-scale colored point cloud (qualitative figure).
- SUN RGB-D: single frames — no reconstruction possible there.
- Own recordings: handheld footage has no poses, so either run [Open3D's RGB-D reconstruction pipeline](#) (RGB-D odometry + pose graph + TSDF), or skip odometry entirely with a feed-forward model like [VGGT](#) or [MASt3R](#) that estimates poses and dense geometry straight from the frames.
- Deliverable: a fly-through room mesh with detected obstacles highlighted (+2–4 pd).

Outputs

- Python pipeline package (loaders, geometry, mapping, eval harnesses).
- Demo videos: annotated RGB beside live BEV map with warnings (KITTI drives, indoor scenes, our walkthroughs).
- Results: two benchmark tables + error-vs-distance plots + sensor-vs-model depth ablation (+ the zero-shot-vs-trained ablation if a Training add-on runs).

2. 360° Surround-View, All-Weather Obstacle Mapping (nuScenes)

Idea: Run the obstacle-mapping pipeline on all six cameras of the [nuScenes](#) rig and fuse the results into a single 360° bird's-eye-view map around the ego vehicle, scored with nuScenes' official center-distance metric and including night and rain conditions KITTI lacks.

Data / Datasets

- [nuScenes-mini v1.0](#): 3.88 GB single archive, **direct public URL, no account** ([v1.0-mini.tgz](#)); 10 scenes, all 6 cameras (1600×900), 3D boxes on 2 Hz keyframes, LiDAR included for depth validation.
- Optional scale-up: free registration (CC BY-NC-SA) + "keyframe blobs only" ≈ 60 GB.
- [nuImages](#) (training only — see Training add-on): 93k 2D-annotated driving images.

Methods / Models

- Same [YOLO11](#) (ONNX) + [Depth Anything V2](#) Outdoor-metric (PyTorch) as idea 1; training: none required.
- [nusenes-devkit](#) (`uv pip install nusenes-devkit`): `get_sample_data()` returns image + ground-truth boxes already in the camera frame + intrinsics; `calibrated_sensor/ego_pose` give camera→ego→global transforms; `map_pointcloud_to_image()` gives LiDAR-projected depth.
- RANSAC ground removal + clustering + fusion, identical to idea 1.

Pipeline

- Preprocess: iterate keyframes; per camera pull image, GT boxes, intrinsics via the devkit; resize for YOLO.
- Per camera: detect → DA-V2 depth → back-project → RANSAC ground removal → cluster → fuse labels.
- Transform each camera's obstacles into the ego frame (`calibrated_sensor` extrinsics).
- Merge the six partial maps into one 360° BEV grid around the car; render with warning zones.
- Class mapping: COCO covers 6 of 10 nuScenes classes (car, truck, bus, pedestrian, motorcycle, bicycle); barriers/cones surface as "unknown obstacle" clusters.

Evaluation

- Tier 1: our own center-distance matching (0.5/1/2/4 m thresholds) per camera frame — nuScenes' matching rule, which scores centroids rather than box IoU, fits a clustering pipeline.
- Tier 2: official [DetectionEval](#) (`detection_cvpr_2019` config) on `mini_val` — needs results in the global frame (camera→ego→global transform chain); earns "evaluated with the official nuScenes protocol."
- Depth check: AbsRel/RMSE of DA-V2 vs LiDAR-projected depth.
- Condition slices: same metrics split day vs night vs rain.

Training add-on (optional — zero-shot stays the baseline row; train on train splits only)

- Option A — fine-tune [YOLO11n](#) on a [nulimages](#) subset (+2-3 pd): 93k 2D-annotated driving images purpose-built for training detectors that transfer to nuScenes (the only role nulimages plays in this project family — useless for our 3D evaluation, ideal for training). Convert 2D boxes to YOLO format, fine-tune ~50 epochs (~2 h GPU), re-run the center-distance evaluation. Ablation: zero-shot COCO detector vs nulimages-tuned detector, same depth and geometry.
- Option B — condition-robustness fine-tune (+1 pd on top of A): train one detector on day-only imagery and one on a mix including night/rain, then compare the night-slice metrics — a direct "does training data diversity buy robustness" result.

3D scene reconstruction add-on (optional)

- Poses are free here: every keyframe carries an `ego_pose`, so transform all six cameras' depth clouds into the global frame and accumulate across a scene — a city-block-scale colored reconstruction.
- Keyframes are 2 Hz, so the result is sparse: treat it as a qualitative figure, not an evaluated output.

Outputs

- 360° BEV map video orbiting the ego vehicle.
- Official-protocol metric table + day/night/rain comparison figure.

- Gotcha handled and documented: nuScenes `Box.center` is the geometric center; KITTI's label is the box *bottom*-center (add $h/2$ when cross-comparing).
-

3. Persistent Room-Scale Obstacle Map (ScanNet / TUM RGB-D)

Idea: Instead of independent per-frame maps, accumulate obstacle evidence across an entire RGB-D walkthrough into one persistent world-frame occupancy map of a room — using the dataset's provided camera poses, so no SLAM is required — that remembers obstacles behind the camera and stays consistent on revisits.

Data / Datasets

- **ScanNet:** 1,500+ room scans — RGB-D streams (`.sens` files, extracted with the [SensReader tool](#)) with **per-frame camera poses** and 3D instance labels on the reconstructed mesh. Requires a signed terms-of-use request (turnaround = days → submit day 1).
- **TUM RGB-D** (instant fallback): Kinect sequences with motion-capture poses; no object labels, so evaluation degrades to occupancy consistency + stop decisions.

Methods / Models

- **YOLO11** (ONNX) + sensor depth; **RANSAC** + clustering as in idea 1; training: none required.
- Log-odds occupancy-grid update (the classic [Elfes formulation](#)): each cell accumulates evidence per frame, so depth noise fades and persistent obstacles solidify.
- Rigid transforms: per-frame camera→world poses from the dataset (ScanNet poses are camera-to-world — verify direction early).

Pipeline

- Preprocess: extract frames + depth + poses from `.sens` (ScanNet) or load sequences (TUM); intrinsics from dataset calibration.
- Per frame: detect → back-project sensor depth → RANSAC floor removal → obstacle points (camera frame).
- Transform points to the world frame via the frame's pose; update the log-odds grid.
- Periodically re-cluster the world-frame map and attach class labels from accumulated detections.
- Render the growing top-down map through the walkthrough.

Evaluation

- Ground truth: project ScanNet's mesh instance labels onto the ground plane → GT occupancy + object list.
- Cell-level occupied/free precision-recall of the final map.
- Object recall: does each labeled chair/table/sofa appear as a cluster within X m of its true position?
- Revisit consistency: map change when the camera re-observes an area (should be ~zero).

- TUM path: stop-decision accuracy ("obstacle within 1 m") vs GT depth, plus map stability.

Training add-on (optional — zero-shot stays the baseline row; train on train splits only)

- Option A — indoor detector fine-tune (+2-3 pd): fine-tune the 2D detector on [SUN RGB-D](#)'s 2D boxes (~2 h GPU) before deploying on ScanNet frames; ablate zero-shot vs indoor-tuned object recall in the final map.
- Option B — learned upper bound without training: compare our classical map against a *pretrained* [VoteNet](#) checkpoint. Training VoteNet ourselves (~overnight) is possible but poor value for the schedule.

3D scene reconstruction add-on (optional — the natural home for it)

- Upgrade the 2D log-odds grid to a 3D TSDF volume ([Open3D](#)) and extract a mesh with marching cubes — the same poses and depth feed both representations.
- ScanNet ships the ground-truth reconstructed mesh, so this is the only idea where reconstruction gets a *number*, not just a picture: report accuracy/completeness (mesh-to-mesh distance) against the GT mesh.

Outputs

- Time-lapse video of the map growing as the camera tours the room (the most visual demo of all 7 ideas).
- Occupancy precision/recall table + per-class object-recall table.
- Reusable accumulation module (drops into ideas 4 and 5 unchanged).

4. Navigation-in-the-Loop (Habitat-Sim) — *our extension #2*

Idea: Close the perception→action loop by having a simulated agent navigate photorealistic indoor scans to point goals using only the obstacle map our own pipeline builds from its RGB-D observations — directly testing whether the map is good enough to navigate with.

Data / Datasets

- Habitat episodes: start/goal pairs in [ReplicaCAD](#) scenes (available instantly, no approval) and [HM3D](#) scans (free access request — submit day 1, switch when it clears).
- Sensor observations per step: RGB, metric depth, ground-truth agent pose.

Methods / Models

- [YOLO11](#) (ONNX) + depth (simulator depth, and [Depth Anything V2](#) on the sim's RGB for the ablation); RANSAC + clustering as in idea 1; training: none required.
- World-frame BEV accumulation from GT pose (idea 3's module; no SLAM).
- [A*](#) planner on our own occupancy grid; discrete action controller (0.25 m forward, 10-30° turns).

- Baselines: built-in [GreedyGeodesicFollower](#) oracle (upper bound), random/straight-line agent (lower bound).

Pipeline

- Setup: conda install [habitat-sim](#) (≤ 1 day; the one fiddly step is EGL/GPU rendering); load scene + episode.
- Loop per step: `sim.step(action)` → RGB-D + pose → detect → back-project → ground removal → cluster → update world-frame BEV grid → A* path to goal on the grid → emit next discrete action → repeat until STOP.
- Record trajectory, map snapshots, and success per episode; repeat over N episodes per scene.

Simulation

- Platform: [Habitat-Sim](#) (all-Python API, modest GPU footprint).
- Why this simulator: photoreal real-building scans, built-in oracle baseline, citable protocol ([Habitat](#), [ICCV 2019](#)) — and every failure mode is in our own Python, so all three of us can debug it.

Evaluation

- Success Rate: fraction of episodes where the agent stops within the task's success radius of the goal.
- SPL (Success weighted by Path Length): $SPL = (1/N) \sum S_i \cdot l_i / \max(p_i, l_i)$ — the [Habitat Challenge](#) protocol (CVPR Embodied AI Workshop 2019–2023), reported against the geodesic oracle.
- Ablation: identical agent with perfect sim depth vs DA-V2 estimated depth → "what does depth quality cost in navigation success," in one chart.

Training add-on (optional — zero-shot stays the baseline row)

- Option A — behavior cloning (+3–4 pd): roll out the [GreedyGeodesicFollower](#) oracle to collect (egocentric BEV-map crop, action) pairs, train a small CNN/MLP policy (hours on GPU), then report a three-way SR/SPL comparison: oracle / our A*-on-map agent / learned policy.
- Option B — detector fine-tune on sim renders (+1–2 pd): only worth it if COCO-pretrained detection visibly underperforms on rendered scenes; labels come free from the simulator's ground-truth semantics.
- Out of scope: full reinforcement learning — [DD-PPO](#)-style PointNav training used ~ 2.5 billion frames of experience — GPU-months, not laptop-hours.

3D scene reconstruction add-on (optional)

- Trivially clean here: ground-truth pose + perfect sim depth → TSDF fusion yields a tidy mesh of the scene as the agent explores.
- Two free experiments: coverage (how much of the scene each episode reconstructs), and the same depth ablation as navigation — reconstruct once from sim depth and once from DA-V2

depth, and measure the mesh degradation against the scene's actual geometry.

Outputs

- Split-screen videos: agent first-person view | growing BEV map with planned path | top-down truth.
 - SR/SPL table (ours vs oracle vs random, + learned policy if trained) + depth-ablation chart.
 - The complete sim harness (reused by idea 7).
-

5. Robot-Stack Deployment (Gazebo + ROS 2 Jazzy + Nav2)

Idea: Package the pipeline as a ROS 2 node so a simulated TurtleBot 4 navigates a Gazebo warehouse using Nav2 whose costmap is fed *only* by our perception (LiDAR disabled) — and the identical node later pointed at a real RealSense becomes the hardware demo for free.

Data / Datasets

- No benchmark dataset — the "data" is the simulated robot's RGB-D stream and a fixed suite of `NavigateToPose` goal runs in the [TurtleBot 4 warehouse world](#).
- Install: `apt install ros-jazzy-turtlebot4-simulator` (official [Jazzy support](#); native on Wade's machine).

Methods / Models

- [YOLO11](#) (ONNX) + sensor (simulated) depth; RANSAC + clustering as in idea 1; training: none required.
- `rcipy` perception node; `PointCloud2` publishing with correct TF frames (topics bridged via [ros_gz_bridge](#)).
- [Nav2 costmap](#) obstacle/voxel layer configured with our topic as the **only** observation source.
- Optional comparison mapper: [RTAB-Map](#) RGB-D SLAM (`apt install ros-jazzy-rtabmap-ros`).

Pipeline

- Bridge Gazebo RGB-D topics to ROS 2 (`ros_gz_bridge parameter_bridge`).
- Node: subscribe RGB + depth → detect → back-project → RANSAC ground removal → cluster → publish surviving obstacle points as `PointCloud2` (+ TF).
- Nav2: costmap marks/inflates our points; planner + controller drive the robot; we send goal poses.
- Tune inflation radius and raytrace/obstacle ranges (known [Nav2 issue](#): marked voxels without replanning).
- Run the fixed goal suite; log success, collisions, and traversal times.

Simulation

- Platform: Gazebo Harmonic + ROS 2 Jazzy + TurtleBot 4 + Nav2 — the industry-standard robotics stack.
- Trade-off: highest real-robotics credibility, but TF/QoS/costmap debugging lands almost entirely on the one ROS-experienced member; the payoff is that sim and hardware demos share one node.

Evaluation

- [BARN-style](#) score per run (ICRA navigation challenge 2022-2026): $s = \text{success} \times \text{OT} / \text{clip}(\text{AT}, 2 \cdot \text{OT}, 8 \cdot \text{OT})$ (OT = optimal traversal time, AT = actual).
- Success rate, collision count, time ratio across the goal suite.
- Baselines: stock Nav2 with its LiDAR enabled (upper bound); optional RTAB-Map-fed costmap comparison.

Training add-on (optional)

- None recommended — Nav2's planner is classical and this idea's substance is systems integration. The nearest honest option is fine-tuning the detector on warehouse-sim renders as a sim-to-real domain-adaptation experiment, but it dilutes the systems story; if the group wants training, pick a different idea's add-on instead.

3D scene reconstruction add-on (optional)

- Essentially free here: [RTAB-Map](#) is an RGB-D SLAM reconstruction system — enable its 3D occupancy/mesh export while it runs as the comparison mapper, and a warehouse reconstruction falls out with no extra code.

Outputs

- Videos of the TurtleBot navigating on perception-only costmaps.
- Scored run table vs the LiDAR baseline.
- A reusable ROS 2 perception node — which *is* the hardware demo when pointed at real RealSense topics.

6. Dynamic Obstacles + Collision Forecasting (RoboWorld-style)

Idea: Add the time dimension: track every moving obstacle with a from-scratch SORT tracker, lift each track to metric 3D through depth, forecast one second ahead with a constant-velocity model, and raise the alarm when a *predicted* — not current — position crosses the ego corridor.

Data / Datasets

- [KITTI tracking benchmark](#) (driving, GT track IDs) and [MOT17](#) (pedestrians, public detections + GT).
- Staged clips: teammates walking toward/across the camera at a measured pace (tripod or cart to avoid ego-motion) — known walking speed + frame count = ground-truth time-to-

contact.

Methods / Models

- [YOLO11](#) (ONNX) detections; training: none required.
- [SORT](#), implemented from scratch: one [Kalman filter](#) per track (state [cx, cy, scale, aspect] + velocities), [Hungarian](#) assignment on an IoU cost matrix, simple track birth/death rules.
- Depth lift (RealSense or [Depth Anything V2](#)) from box footpoint → metric 3D track positions.
- Constant-velocity forecast at a 1 s horizon; ego-corridor intersection test for alarms.

Pipeline

- Per frame: detect → Hungarian-match detections to predicted track boxes → Kalman update matched tracks, spawn/kill unmatched ones.
- Lift each active track to 3D via depth at its footpoint.
- Forecast each track 1 s ahead; flag tracks whose predicted path enters the ego corridor.
- Render: BEV map with track trails, predicted paths, and alarm state; alarm log with timestamps.

Evaluation

- Tracking: MOTA, IDF1, ID switches on KITTI-tracking/MOT17 via the official eval code or [py-motmetrics](#).
- Forecasting: ADE/FDE at 1 s against each track's own realized future positions.
- Alarms: precision, recall, and *latency* (frames of warning before the true threshold) on the staged clips with measured-speed ground truth.

Training add-on (optional — the classical tracker stays the baseline row; train on train splits only)

- Option A — SORT → [DeepSORT](#) (+3–4 pd): train a small Re-ID embedding CNN on MOT17 identity crops with a triplet loss (a few hours on GPU) and add appearance distance to the association cost. Crisp ablation: ID switches and IDF1, SORT vs DeepSORT, same detections.
- Option B — learned forecaster (+2 pd): train an MLP trajectory forecaster on past-position windows and ablate it against the constant-velocity model on ADE/FDE — tiny model, direct improvement claim.

3D scene reconstruction add-on (optional)

- The interesting twist here: mask tracked movers out of TSDF fusion and reconstruct only the static background — "static scene reconstruction with dynamic-object removal," which ties the tracker directly into the reconstruction and demos well (people walk through the scene; the mesh stays clean).

Outputs

- Overlay videos: tracks with IDs, predicted paths, and firing alarms.
- MOT metric table + forecast ADE/FDE table + alarm precision/recall/latency table.

- On-ramp note: this variant is shaped like the RGB-D safe/social-navigation tracks of the [RoboWorld Challenge \(NeurIPS 2026\)](#) — a realistic post-course entry with the same code.
-

7. Language-Directed Safe Navigation (CMU-VLN-style)

Idea: Let a typed instruction — "go to the whiteboard by the door, keep clear of the cart" — choose the goal: an open-vocabulary detector grounds the named object in the frame, our pipeline places goal and obstacles in one metric BEV map, and a planner produces a collision-free path to the named object.

Data / Datasets

- Our own RealSense recordings with goal objects at tape-measured positions, plus a fixed instruction set (including paraphrases and distractors — "the chair" with three chairs visible).
- Habitat episodes reusing idea 4's harness ([ReplicaCAD](#) / [HM3D](#) scenes with known object placements).

Methods / Models

- [YOLO-World \(CVPR 2024\)](#) via the ultralytics package: open-vocabulary detection from text prompts; `model.set_classes([...])` precomputes the text embeddings per scene vocabulary. Training: none required.
- Rule-based instruction parsing: extract goal phrase + avoid phrases (kept deliberately simple).
- Idea 1's spine (depth, RANSAC, clustering, fusion) for the obstacle side; [A*](#) with obstacle inflation zones for planning.

Pipeline

- Parse the instruction → goal phrase + avoid list → set the detector's open vocabulary.
- Detect the goal object and standard obstacles; estimate depth; back-project; remove ground; cluster; fuse.
- Place the goal cell and all obstacles (inflated) in the BEV grid.
- [A*](#) from the camera/agent position to the goal cell avoiding inflated obstacles → waypoints.
- In Habitat: execute waypoints as discrete actions (idea 4's controller). On recordings: render the planned path overlaid on the map and image.

Simulation

- [Habitat-Sim](#), reusing idea 4's loop and metrics — this idea is only sensible *on top of* a working idea 4.

Evaluation

- Grounding accuracy: was the correct object selected for each instruction?
- Goal localization error vs tape-measured positions.

- Navigation: success rate + SPL over the instruction set in Habitat (success = stop within radius of the *named* object without collision).
- Stress analysis: failure modes on paraphrases and distractor-heavy scenes, reported honestly.

Training add-on (optional — zero-shot grounding stays the headline; train on train splits only)

- Option A — few-shot fine-tune of [YOLO-World](#) on a small hand-labeled set of our own lab objects for the distractor-heavy cases (+3 pd). Caution: fine-tuning partly fights the open-vocabulary premise — keep zero-shot grounding as the headline result and frame the fine-tune as a robustness experiment only.

3D scene reconstruction add-on (optional)

- Reconstruction + open-vocabulary detections = a queryable semantic 3D map: fuse the mesh (idea 4's TSDF path), attach grounded labels to mesh regions, and typing "whiteboard" highlights it in the room model — the [VLFM](#) direction in miniature.

Outputs

- Demo videos: typed instruction → grounded goal → planned path → executed navigation.
- Grounding/localization table + success/SPL table + failure-mode analysis.
- Context note: this is the task family of the [CMU VLN Challenge \(IROS 2026\)](#) (registration free until Jul 25, 2026; submission Aug 25 — post-course only), and the variant closest to Thomas's and Wade's VLM interests.

How these combine

Idea 1 is the committed core; idea 4 is extension #2 in the proposal. Ideas 2, 6, 7 are add-ons in cost order (2 is cheapest; 7 requires 4). Ideas 3 and 5 are alternative *shapes* of the project rather than add-ons — 3 trades benchmark breadth for mapping depth; 5 trades schedule safety for robotics-stack credibility plus a nearly-free hardware demo. If RealSense access falls through, ideas 1+2 run fully monocular (KITTI + nuScenes-mini) with zero hardware. Training is never on the critical path: every Training add-on is an *ablation against the zero-shot baseline*, trained only on train splits, so a failed training run costs an experiment, not the project.